Optimistic Register Coalescing

JINPYO PARK and SOO-MOOK MOON
Seoul National University

Graph-coloring register allocators eliminate copies by coalescing the source and target nodes of a copy if they do not interfere in the interference graph. Coalescing, however, can be harmful to the colorability of the graph because it tends to yield a graph with nodes of higher degrees. Unlike *aggressive coalescing*, which coalesces any pair of noninterfering copy-related nodes, *conservative coalescing* or *iterated coalescing* perform safe coalescing that preserves the colorability. Unfortunately, these heuristics give up coalescing too early, losing many opportunities for coalescing that would turn out to be safe. Moreover, they ignore the fact that coalescing may even improve the colorability of the graph by reducing the degree of neighbor nodes that are interfering with both the source and target nodes being coalesced. This article proposes a new heuristic called *optimistic coalescing* which optimistically performs aggressive coalescing, thus exploiting the positive impact of coalescing aggressively, but when a coalesced node is to be spilled, it is split back into separate nodes. Since there is a better chance of coloring one of those splits, we can reduce the overall spill amount.

Categories and Subject Descriptors: D.3.4 [**Programming Languages**]: Processors—*code generation, optimization*

General Terms: Algorithms, Languages

Additional Key Words and Phrases: Copy coalescing, graph coloring, register allocation, noncopy coalescing

1. INTRODUCTION

Many optimizing compilers take the approach of *graph coloring* for global register allocation [Chaitin 1982]. Graph coloring abstracts the problem of assigning registers to live ranges in a program into the problem of assigning colors to nodes in an *interference graph*. The register allocator attempts to "color" the graph with a finite number of machine registers, with one constraint that any two nodes connected by an interference edge must be colored with different registers. If the allocator fails to color the graph, some nodes must be *spilled* to memory by inserting loads and stores. Coloring the sgraph with minimum spills

Authors' address: School of Electrical Engineering and Computer Science, Seoul National University, San 56-1 ShinLim-Dong, KwanAk-Gu, Seoul 151-742, Korea; email: smoon@altair.snu.ac.kr.
Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or direct commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 1515 Broadway, New York, NY 10036 USA, fax: +1 (212) 869-0481, or permissions@acm.org.
© 2004 ACM 0164-0925/04/0700-0735 $5.00

is a well-known NP-complete problem, so many heuristic approaches have been used [Chaitin 1982; Briggs et al. 1994; Chow and Hennessy 1990].

One important task of a register allocator is copy propagation that eliminates unnecessary copies. In the context of graph coloring, copy propagation is achieved simply by coloring the source and target nodes of a copy (which we call *copy-related nodes*) with a single register if they do not interfere. In the interference graph, this is achieved by *coalescing* the two nodes into a single node, with their interference edges being unioned. Many optimization phases including instruction scheduling [Moon and Ebcioğlu 1997], store-to-copy promotion [Cooper and Lu 1997], and static single assignment (SSA) translation [Cytron et al. 1991] leave behind many copies that would slow down program execution and increase code size; hence it is essential to remove as many of them as possible to produce quality code.

On the other hand, coalescing may affect the colorability of the interference graph. Since a coalesced node will have the union of interference edges of the nodes being coalesced, it might not be possible to color the coalesced node. This makes the graph coloring register allocation problem more challenging because we need to minimize the spill amount while maximizing the number of coalesced nodes. A couple of coalescing heuristics have been proposed to handle this optimization problem.

The original register allocator by Chaitin [1982] uses *aggressive coalescing* which coalesces any pair of noninterfering, copy-related nodes. It obviously achieves the best amount of copy elimination but might suffer from worst spill. Briggs et al. [1994] have devised a heuristic called *conservative coalescing*, which performs coalescing only when the colorability of the graph is not affected. George and Appel's [1996] *iterated coalescing* eliminates more copies than conservative coalescing by interleaving Briggs' heuristic with a coloring simplification phase, which increases the opportunity for conservative coalescing.

While conservative coalescing and iterated coalescing attempt to avoid the negative impact of coalescing, it is often ignored that coalescing may even improve the colorability of the graph: if a node x interferes with both the source and target nodes being coalesced, the degree of x is reduced by one after the coalescing. This may yield additional opportunities for the simplification of the graph, improving the overall colorability. In order to aggressively exploit this positive impact of coalescing, it is preferable to employ aggressive coalescing, except that spills must be controlled somehow.

This article proposes a new coalescing heuristic called *optimistic coalescing*, which optimistically coalesces all pairs of noninterfering, copy-related nodes as in Chaitin's heuristic, but when a coalesced node needs to be spilled, the node is split back into separate nodes, giving up the coalescing. Since each split node will have fewer interference edges than the original coalesced node, it might be possible to color some of them without affecting the rest of the coloring, which reduces the overall spill amount.

This article is structured as follows. Section 2 briefly reviews graph-coloring register allocation. Section 3 describes previous coalescing heuristics. Section 4 introduces optimistic coalescing and details several implementation issues. An extended version of optimistic coalescing is described in Section 5. The

experimental results are presented in Section 6. A discussion on live range splitting and precoloring can be found in Section 7 and a summary follows in Section 8.

2. GRAPH-COLORING REGISTER ALLOCATION

Graph coloring is a problem of assigning colors to nodes of a graph such that any two nodes connected by an edge have different colors. In graph-coloring register allocation, each node in the graph corresponds to a live range in the program and the number of colors represents that of machine registers. Two overlapping live ranges are said to be interfering and the corresponding nodes are connected by an edge.

Since coloring a general graph G with K colors is NP-complete, we need a heuristic algorithm. Chaitin et al. [1982] have implemented graph-coloring register allocation based on the following theorem:

A node x having fewer than K neighbors is always colorable no matter how G-$\{x\}$ is colored.

Such a node x is called a *low-degree* node while a node that has K or more neighbors is called a *significant-degree* node. Chaitin's allocator repeatedly removes low-degree nodes and their associated edges from the graph and pushes the nodes on top of the stack until either the graph is empty or every node in the graph is significant. This is called the *simplify* phase. If the graph is empty, the allocator repeatedly takes nodes back from the top of the stack and assigns appropriate machine registers that obey interference constraints. This is called the *select* phase. If every node is significant in the simplify phase, however, the allocator heuristically chooses nodes and marks them for spills. In the *spill* phase, the allocator actually spills each marked node by inserting loads and stores at uses and definitions of the corresponding live range, respectively, and the register allocation repeats from scratch.

The phase ordering of Chaitin's coloring algorithm is depicted in Figure 1(a). The *renumber* phase and the *build* phase are for constructing live ranges and for building the interference graph, respectively (the phase of *aggressive coalescing* will be described in the next section).

In order to reduce the number of spilled nodes, Briggs et al. [1994] have introduced *optimistic coloring*. When every node is significant, instead of marking a node for spilling, the allocator just removes and pushes it on the stack. This is called *potential-spill*. During the select phase, if there indeed is no color for that node, it is *actually-spilled*. The idea is that colors may be available even for significant-degree nodes depending on the coloring of their neighbors, but this opportunity is pessimistically given up too early in Chaitin's algorithm. Figure 1(b) shows the phase ordering of optimistic coloring.[1]

[1]The original flow graph of Briggs' optimistic allocator includes the *rematerialization* phase [Briggs et al. 1994] which generates copies to be removed by conservative coalescing (see Section 3.2.1). It is omitted in our version of the optimistic allocator to evaluate conservative coalescing as a general coalescing heuristic, not bound to a specific optimization.

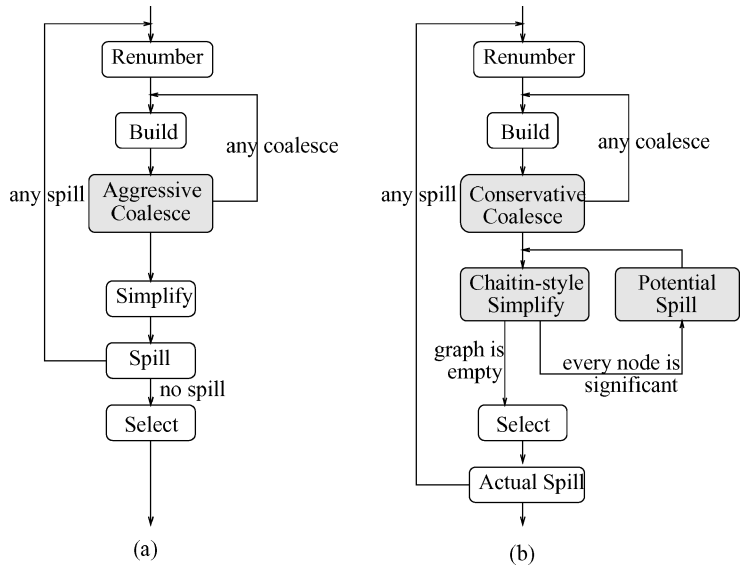

Fig. 1. Phase ordering of (a) Chaitin's allocator and (b) Briggs' optimistic allocator.

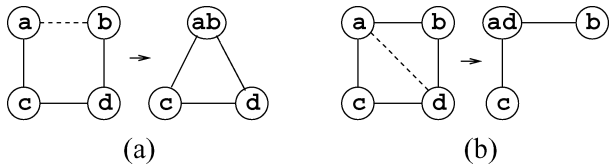

Fig. 2. Examples of (a) the negative impact and (b) the positive impact of coalescing.

3. APPROACHES TO REGISTER COALESCING

Coalescing has both negative and positive impacts on the colorability of the interference graph. When two nodes are coalesced, the coalesced node will have a union of the edges of those being coalesced. This can make a K-colorable graph not K-colorable anymore. On the other hand, if a node interferes with both nodes being coalesced, the degree of the node is reduced by one which may expose an additional opportunity for simplifying the graph, improving the overall colorability.

For example, consider the problem of coloring the interference graphs in Figure 2 with two colors ($K = 2$) where a solid line represents an interference edge and a dotted line represents a copy-related edge. If a and b are coalesced as in Figure 2(a), the original graph becomes a graph that is not two-colorable any more. In Figure 2(b), however, a graph that cannot be simplified with two colors becomes simplifiable after coalescing a and d, making the graph two-colorable with Chaitin's algorithm.

There have been two approaches to coalescing. One is eliminating all coalescible copies, aggressively exploiting the positive impact. Chaitin's aggressive coalescing corresponds to this. The other focuses on reducing the negative impact, which includes conservative coalescing and iterated coalescing.

3.1 The Approach of Aggressive Coalescing

Chaitin's allocator coalesces any noninterfering, copy-related nodes. From the perspective of copy elimination, aggressive coalescing achieves the best result. From the perspective of the colorability of the graph, it aggressively exploits the positive impact while totally ignoring the negative impact. Unfortunately, the advantage of the positive impact in aggressive coalescing has never been noted previously, and it has never been questioned how the positive impact interacts with the negative impact, affecting the overall colorability of the graph. When aggressive coalescing is employed in Briggs' optimistic allocator, we empirically found that the positive impact well outweighs the negative impact, coloring the graph better than conservative coalescing or iterated coalescing (see Section 6).

3.2 The Approach of Conservative Coalescing

3.2.1 *Conservative Coalescing.* Briggs et al. [1994] have proposed an optimization phase called *rematerialization* on top of their optimistic register allocator. It reduces the spill cost of a live range if its value is cheaply reproducible from a constant. Finding rematerializable values involves an SSA transformation which generates copies to split live ranges into rematerializable sub live ranges. These copies, unlike other copies that can be coalesced aggressively, must be coalesced "conservatively" because reckless coalescing would completely restore the transformation and if the coalesced node is spilled, we lose the chance of cheap spilling. This has motivated the heuristic of conservative coalescing which is based on the following theorem [Briggs et al. 1994]:

For two given nodes x *and* y*, if the coalesced node* xy *has fewer than K significant-degree neighbors,* xy *does not change the colorability of the original interference graph*.

If all low-degree nodes are removed from the graph, the coalesced node xy can surely be removed since it originally had fewer than K significant-degree neighbors (i.e., for a node to be spilled, it must have at least K significant-degree neighbors originally). So, a conservatively coalesced node is never spilled.

We must be careful in counting the number of significant-degree neighbors of xy because a simple union of significant-degree neighbors of x and y may include nodes that become low-degree after coalescing (due to the positive impact). Let S_x and S_y be the sets of significant-degree neighbors of x and y, respectively. Then, the set of significant-degree neighbor of xy, S_{xy}, is computed as follows: $S_{xy} = S_x \cup S_y - \{z | z \in S_x \cap S_y,\ \text{degree of } z = K \text{ before the coalescing}\}$.[2]

Briggs' allocator with rematerialization performs two rounds of coalescing: aggressive coalescing of copies other than split copies, followed by conservative coalescing of split copies. In order to complement conservative coalescing, *biased coloring* is used in the select phase which removes additional copies by attempting to assign the same register to copy-related nodes that have not been coalesced.

[2]It appears that the iterated coalescing algorithm in George and Appel [1996] ignores this term in computing the number of significant-degree neighbors, failing to perform conservative coalescing fully. In our implementation, we included it for a fair comparison.

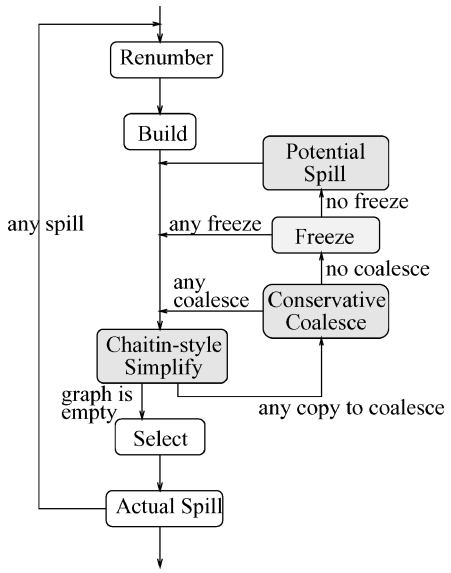

Fig. 3. Phase ordering of iterated coalescing on top of optimistic allocator.

In this article, we evaluate conservative coalescing as a general coalescing heuristic, not bound to a specific optimization. Biased coloring is also included in our implementation of conservative coalescing.

3.2.2 *Iterated Coalescing*. Generally, applying both aggressive coalescing and conservative coalescing may complicate the coalesce phase. For example, the order in which to apply both heuristics may be somewhat arbitrary and intermixing both heuristics might not prevent additional spills from being introduced [Briggs et al. 1994].

George and Appel [1996] introduced iterated coalescing where they eliminated aggressive coalescing completely (partly because it overconstrains precoloring in their SML/NJ compiler [Appel and MacQueen 1991]; see discussion in Section 7). Iterated coalescing coalesces more copies conservatively. It performs conservative coalescing iteratively, interleaved with the simplify phase which exposes more chances for conservative coalescing. Figure 3 depicts the phase ordering of iterated coalescing on top of Briggs' optimistic allocator.

In iterated coalescing, the allocator maintains two groups of nodes. One group is a set of copy-related nodes and the other is its complementary set composed of non-copy-related nodes. Simplifying low-degree, non-copy-related nodes will decrease the degree of many copy-related nodes, yielding more opportunities for conservative coalescing. A coalesced node is put into the group of non-copy-related nodes if it has no more copy-related neighbors. When all non-copy-related nodes are significant-degree and no two copy-related nodes are coalescible any more, the allocator *freezes* one of the low-degree, copy-related nodes, if there are any. Freezing a copy-related node means that we give up its coalescing by removing all of its copy-related edges and putting it into the group of non-copy-related nodes. A frozen node may be simplified in the next

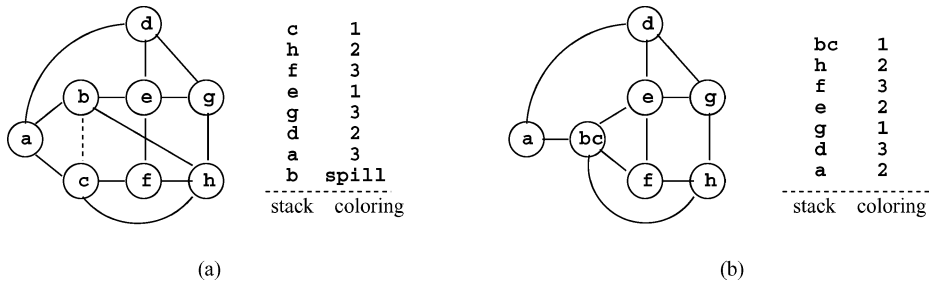

Fig. 4. Allocation with (a) conservative coalescing and (b) aggressive coalescing.

iteration. When all nodes in both groups are significant-degree, a potential spill is made and the iteration of simplification and conservative coalescing continues. Biased coloring is also included in the select phase in our implementation of iterated coalescing.

3.2.3 *Problems with the Approach of Conservative Coalescing.* The problem with the approach of conservative coalescing is two-fold. First, the heuristic gives up coalescing too early in the sense that a coalesced node that violates the conservativeness heuristic is not necessarily spilled. This is somewhat similar to the motivating idea of Briggs' optimistic allocator that a potentially-spilled node is not necessarily actually-spilled. Just as he delayed the spill decision to later, we can also delay any consideration for spills in coalescing to later, if many coalesced nodes that violate the heuristic are not actually-spilled and if there is a way to lower the spill cost in case they are actually-spilled (Section 6.3 shows experimental data that justify the delay of spill considerations).

Secondly, the heuristic does not exploit the positive impact of coalescing as much. Consider the interference graph in Figure 4(a). Let us assume that three colors are available ($K = 3$) and `b` is most preferred for potential spilling. In this graph, neither conservative coalescing nor iterated coalescing can coalesce `b` and `c`, so `b` is potentially-spilled. Once `b` is removed from the graph, other nodes can be simplified on the stack as shown in the right. In the select phase, every node is assigned a color (as shown in the right column of the stack) except for `b` which must be actually-spilled (if `e` were colored with 2 not with 1, every node would be assigned a color, but this is not always guaranteed).

On the other hand, aggressive coalescing of `b` and `c` yields a low-degree node `a` as in Figure 4(b) and allows every node in the graph to be simplified (a legal coloring without any spill is shown in the right column of the stack). The question is how often these opportunities arise in real programs.

4. OPTIMISTIC COALESCING

Aggressive coalescing totally ignores the spill of coalesced nodes although the positive impact of coalescing is aggressively exploited and the maximum number of copies are eliminated. On the other hand, conservative coalescing and iterated coalescing prohibit the spilling of coalesced nodes, while losing many opportunities for safe coalescing. The coalescing approach described here, called *optimistic register coalescing*, eliminates almost the same number of copies as

aggressive coalescing, yet reduces the spilling of coalesced nodes substantially, outperforming other coalescing heuristics in our experiments.

4.1 The Idea

In order to aggressively exploit the positive impact of coalescing, we take the same approach of aggressive coalescing as Chaitin and implement it on top of Briggs' optimistic coloring register allocator to reduce spills. During the coalesce phase, the degree of a node may decrease due to the positive impact, while a coalesced node may have more than K significant-degree neighbors. Then the graph is simplified as in Briggs' optimistic allocation such that potential spills are made. When a coalesced node is to be actually-spilled in the select phase, however, we attempt to reduce the spill cost of the node by *undoing* the coalescing. That is, if a coalesced node `xy` needs to be actually-spilled, we split it back into `x` and `y` while recovering the original interference edges of each. It might now be possible to color `x` or `y` or both, because the degree of each node is likely to be lower than that of `xy`.

There are three possible cases for the coloring of the two splits, `x` and `y`.

—Neither `x` nor `y` can be colored. In this case, we mark both of them for the actual spill (this is equivalent to spilling the original node `xy`).

—`x` can be colored while `y` cannot (or vice-versa). In this case, we mark `y` for the actual spill, and assign the color to `x`.

—Each of `x` and `y` can be colored with a different color. In this case, we can color both `x` and `y`. However, assigning different colors to them at this point of the select phase might ruin the rest of the coloring: if a node below `xy` in the stack has been simplified based on the fact that `x` and `y` are not distinct (e.g., a common neighbor node of `x` and `y` whose degree has been reduced after their coalescing, thus being simplified), the node might not be colorable anymore because it now has more colored neighbors. Consequently, it is better to color only one node (preferably a node with the higher spill cost) without affecting the rest of the coloring. We do not have to give up the coloring of the other node, though. Instead of marking the uncolored node for an actual spill at this point, we can try again to find a color for it after processing all nodes left in the stack; if a color is still available for the node at that point, it is safe to color the node.

For example, consider again the interference graph in Figure 4(a), where we add a new interference edge between `e` and `h`. Again, there are three colors ($K = 3$) and `b` is assumed to be most preferred for potential spilling. The interference graph after optimistic coalescing is shown in Figure 5(a). After `a`, `d`, and `g` are simplified, a node needs to be potentially spilled, unlike previously. If the coalesced node `bc` is chosen for the potential spill, the stack would look like in Figure 5(b) after the simplification is done.

In the color select phase, `h`, `f`, and `e` are colored with `1`, `2`, and `3`, respectively. Because no color is available for the next popped node `bc`, it is split back into `b` and `c`. It is possible to color `b` and `c` with `2` and `3`, respectively, but we will color only `c` which is more expensive. The final coloring result is shown in Figure 5(d)

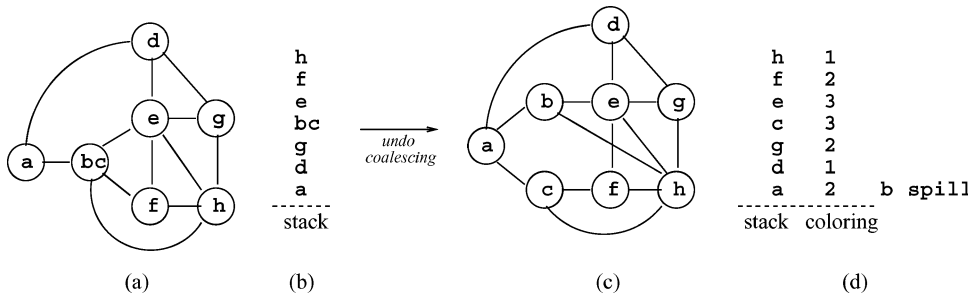

Fig. 5. An example of undoing coalescing.

where b is actually spilled. It should be noted that if we had colored both b and c greedily, we would need to actually spill a (which is more expensive) instead of b.

When we have to split a coalesced node xyz, we first split it into three primitive nodes x, y, and z, and check if each can be colored individually; those nodes that cannot be colored are spilled right away at this point. For the remaining colorable nodes, we make every possible combination of splits and check if each split is colorable with the same color. Among those colorable splits, we choose the best one which minimizes the overall spill cost.

For example, if all of x, y, z can be colored individually, we test all combination of splits: x, y, z, xy, yz, zx, xyz, and {} (meaning spilling all of them). Let us assume that the split x is colorable and is associated with the minimum cost (i.e., the cost of spilling y and z is the minimum). Only x is colored at this point while y and z are set aside for a coloring decision later. After all nodes below the stack are processed, one of the following occurs for y and z: (1) both are colored with a single color, or (2) they are colored differently, or (3) one is colored while the other is spilled, or (4) both are spilled. If x, y, and z are colored with different colors, it means that coalescing is completely undone, with all of the original copies being recovered.

The same procedure is applied when splitting a coalesced node with more than three nodes. The phase ordering of optimistic coalescing on top of the optimistic register allocator is shown in Figure 6.

4.2 Implementation Issues

This subsection addresses some of the issues in implementing optimistic coalescing including the choice of nodes to spill, undoing of coalesced nodes, and coloring of split nodes. We also discuss the algorithm complexity. The detailed pseudo code of optimistic coalescing can be found in Appendix A.

4.2.1 *Spill Preference.* We use a general spill preference metric for potential spill, which is estimated by the spill benefit (based on the degree of nodes) divided by the spill cost (based on the number of loads/stores to be inserted) [Chaitin 1982]. The spill cost of a coalesced node is estimated to be smaller than the sum of the spill costs of its "ingredient" nodes since we can save the cost of the deleted copy itself. When the estimated spill preferences for two nodes are

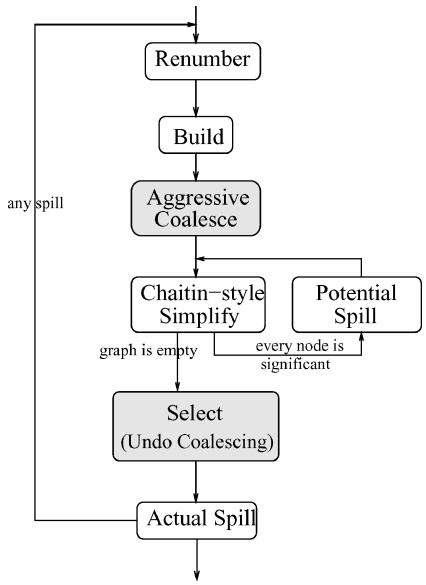

Fig. 6. Phase ordering of optimistic coalescing on top of optimistic allocator.

the same, we prefer a coalesced node to a primitive node since we can reduce the spill amount through undoing of coalescing. If there is more than one candidate coalesced node for the potential-spill, the one with more ingredient nodes is preferred because it has a better chance of being colored after the split (e.g., a coalesced node xyz that is composed of three nodes is preferred to uv that is composed of two nodes).

4.2.2 *Coalescing and Undo of Coalescing.* We should implement coalescing in considering of efficient undo of coalescing. That is, we should be able to extract the primitive nodes from a coalesced node efficiently, with their neighbor node lists and spill costs being recovered.

Coalescing of two nodes is implemented as one node (absorber) absorbing the other node (absorbee). We add absorbee's neighbor nodes into absorber's neighbor node list. Similarly, we add the absorber into the neighbor node list of its newly added neighbors. Instead of deleting the absorbee from its neighbors' neighbor node list and deleting the absorbee itself, we just mark the absorbee inactive and define the degree of a node equal to the number of active neighbors.

We always append new neighbors at the end of a neighbor node list and maintain a single pointer that points the beginning of all newly added neighbors. In this way, we can easily separate all newly added neighbors from the original neighbors. Moreover, we can traverse the original neighbors, the newly added neighbors, and the entire neighbors efficiently.

When we undo coalescing, we should fix the interference graph as if the primitive nodes had not been coalesced. The neighbor node list of a noncoalesced node changes during the coalesce phase as follows. Before the coalesce phase starts, the node interferes with its original neighbors. During the coalesce

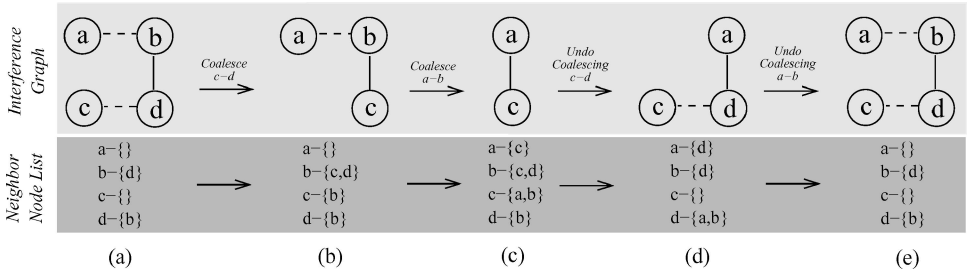

Fig. 7. Example of coalescing and undoing coalescing.

phase, some of its neighbors are absorbed and thus the absorbers of those neighbors are added to the neighbor node list of the node. Therefore, using the original neighbors of a primitive nodes, we can reconstruct its neighbor node list as if it had not been coalesced.

This is achieved in three steps. First, we recover the original neighbor node list of each primitive node by removing the interference between the primitive node and each of its newly added neighbors. Secondly, we construct the correct neighbor node list of each primitive node by adding the interference between the primitive node and the absorber of each of its original neighbors. Finally, we mark each primitive node active if it is not.

Figure 7 illustrates how the interference graph and the neighbor node lists change as we do and undo coalescings. Below each interference graph is the neighbor node list for each node. Figure 7(a) is the original interference graph and Figure 7(b) shows the result after coalescing `c` and `d` with `c` being the absorber. In Figure 7(b), the neighbor node list of `c` gets `b`, the neighbor of `d`. It should be noted that `d` is not removed from `b`'s neighbor node list, but is just marked inactive. Similarly, coalescing `a` and `b` yields the result in Figure 7(c) if `a` is the absorber.

If we split `c` into `c` and `d`, we take the three steps of undoing coalescing. In the first step, we remove the newly added interferences `c`–`a` and `c`–`b`. We do nothing for `d` since it has no newly added neighbor. The neighbors of `a`, `b`, and `c` after the first step are {}, {`d`}, and {}, respectively. In the second step, a new interference between the absorbee `d` and `a` (which is the absorber of `b` which is `d`'s original neighbor) is added. Since the absorber `c`'s neighbor is none, we do not take any action for it. Finally, `d` is marked active. The result is depicted in Figure 7(d). Similarly, undoing the coalescing of `a` and `b` results in Figure 7(e), restoring back to the original interference graph and the neighbor node lists.

Regarding the spill costs, we save the original spill costs before coalescing which are restored after the undo of coalescing easily.

4.2.3 *Coloring Splits.* After we obtain primitive nodes from an undo-coalesce candidate node, we actually-spill those uncolorable primitive nodes right away. Then, we test the colorability of every combination of the colorable primitive nodes as described in Section 4.1. Although this exhaustive test guarantees to find the optimal colorable split that minimizes the overall spill cost, its time complexity is exponential, that is, $O(2^n)$ where n is the number of

colorable primitive nodes in an undo-coalesce candidate node. Fortunately, our experiments show that the average of n is 2.2, so the test will be done quickly in practice. However, the peak of n reaches up to 13, and in theory, it can be any big number. We can reduce the overhead by avoiding redundant tests (e.g., we do not have to check if `xyz` is colorable when it is already known that `xy` is not colorable), but the time complexity is still exponential.

We have a heuristic solution to handle this problem whose time complexity is $O(n^2)$ where n is the number of primitive nodes in an undo-coalesce candidate node. The heuristic algorithm proceeds as follows. For each color C, the algorithm attempts to find the best split that can be colored with C. The best split means the one whose spill cost (the spill cost when its remaining uncolored split is spilled) is the minimum. Among the best colorable splits for each color, we choose the one with the minimum cost as the final colorable split.

When we try to find the best split for a given color, there is an issue to be considered: even when a node is colorable, it might be better to spill it instead of coloring it for some cases. For example, consider a loop that contains a copy `y=x` inside. There is a single definition of `x` just before entering the loop and there is a single use of `y` just after exiting the loop. Let us assume that `y` is not colorable while `x` can be colored. If we color `x`, we need to keep the copy and generate spill code for `y` inside the loop (the copy will be removed eventually since `y=x`; `st y,@stack` will be replaced by `st x,@stack`). If we decide to spill `x` as well as `y`, however, we are actually spilling the whole coalesced node `xy`, which will obviate the copy, hence the spill code inside the loop.

In order to exploit this opportunity, we need to check if spilling a colorable node is beneficial when it has a copy-related edge with a node (or a split) that will be spilled. Our algorithm checks this opportunity heuristically as follows. For a given undo-coalesce candidate node, we build a *copy graph* whose nodes correspond to primitive nodes in the candidate node and whose edges are copy-related edges among the primitive nodes. Then for a given color, we first find a set of nodes that are uncolorable with it. Taking a node in this set as a "seed" node, we traverse the graph in depth-first search (DFS) order from the seed node and compute a DFS tree. When we visit a node along a copy edge, we add the node in the current DFS tree and estimate the overall spill cost of spilling all nodes in the new DFS tree. After the traversal completes, we find the DFS tree that achieves the minimum spill cost. Then we remove all nodes of that DFS tree from the copy graph. We repeat this for each seed node in the set (in decreasing order of spill costs) until no seed node is left in the copy graph. The nodes left in the copy graph after this process is over comprise the colorable split for the given color.

Our evaluation of this heuristic testing reveals that it finds the same optimal colorable split of the exhaustive testing for more than 96% of undo-coalesce candidate nodes. The time overhead of the heuristic testing grows very slowly compared to the exhaustive testing, although the heuristic testing is slower than the exhaustive testing when the number of primitive nodes, n, is small. We can exploit this result by deciding some threshold number T such that if n is less than T, we use the exhaustive testing but if n is bigger than or equal

to T, we use the heuristic testing. The pseudo code of the heuristic testing and the evaluation results are available upon request. We use only the exhaustive testing for the experiments performed in this paper.

4.2.4 *Algorithm Complexity*. In practice, the spatial and temporal complexity of the above algorithm is low. Regarding the spatial overhead, optimistic coalescing additionally requires n words for pointers into the neighbor node lists that separate original neighbor nodes and newly added neighbor nodes, and n words for saved spill costs, where n is the number of nodes. As to the temporal overhead, the testing of colorability can be done quickly as described above. In fact, the timing overhead of undoing coalescing can be worthwhile if it can eliminate all actual-spills, which will then obviate additional iterations of the allocation phases.

5. EXTENDED OPTIMISTIC COALESCING

We can extend optimistic coalescing to even non-copy-related nodes in order to exploit the positive impact further, while mitigating the negative impact. The negative impact of this *extended optimistic coalescing* is greater than that of optimistic coalescing since no copies are removed, so the candidate for coalescing must be chosen carefully.

In our experiments, we employed *merge-enhanced graph coloring* (MEGC) algorithm proposed by Vegdahl [1999] to coalesce non-copy-related nodes. In MEGC when simplification is blocked, non-copy related nodes are coalesced to exploit the positive impact, which might lead to additional simplification of the graph. So, the simplify phase and the coalesce phase of non-copy-related nodes are interleaved until no simplification is possible and no coalescing candidate is available. Potential spill is made at that time. Figure 8 depicts the phase ordering of extended optimistic coalescing using MEGC. During the select phase, we undo the coalescing for non-copy-related nodes as well as for copy-related nodes.

The candidate non-copy-related nodes for coalescing in MEGC are chosen by "pair-score", which estimates the gain of coalescing for a pair of nodes. It is computed by dividing the number of neighbors interfering with both nodes (positive impact) by the number of neighbors of the smaller-degree node (negative impact); for example, a pair-score of 0.0 and 1.0 indicates that the pair has no common neighbors (i.e., negative impact only) and that the neighbors of a node are a subset of those of the other node (i.e., positive impact only), respectively. The pair with higher pair-score is preferred for coalescing.

It takes a long time to compute the pair-score of every node pair in the interference graph and keep it up-to-date as simplification and coalescing change the graph. Vegdahl proposed a faster and effective heuristic that determines the subset of the nodes for the candidate of coalescing. First, it finds M nodes with a small degree that are likely to be simplified due to the positive impact. It then finds top K nodes that are interfering with as many small degree nodes. Those K nodes are candidates for coalescing pair. In our experiments, the heuristic constants M and K are chosen to be 50 and 20, respectively.

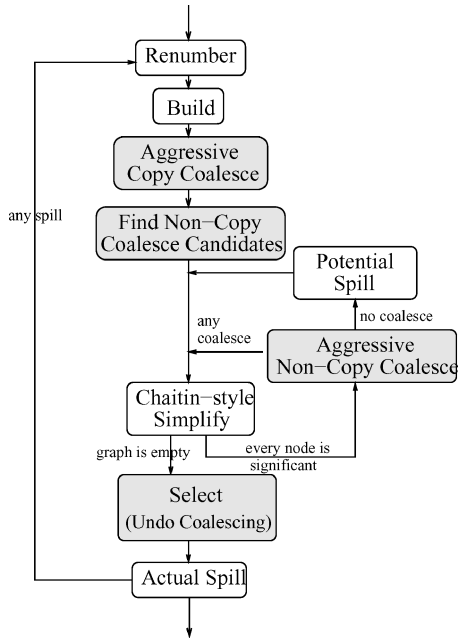

Fig. 8. Phase ordering of extended optimistic coalescing using merge-enhanced graph coloring on top of optimistic allocator.

6. EXPERIMENTAL RESULTS

In order to evaluate how effective optimistic coalescing and extended optimistic coalescing are compared to previous coalescing heuristics, we have performed an empirical study in the context of instruction scheduling on nonnumerical code. The register allocation problem in this context is highly challenging because aggressive code scheduling in the presence of complex control flows generates many copies and increases the register pressure. We have experimented with five coalescing heuristics in this environment.

6.1 Experimental Environment

The experiments have been performed on a SPARC-based VLIW testbed [Moon et al. 1998]. The input C code is compiled to optimized SPARC assembly code (without register windows) by the *gcc* compiler. The SPARC-based assembly code is scheduled to high-performance VLIW code by a software pipelining technique called *enhanced pipeline scheduling* (EPS) [Moon and Ebcioğlu 1997]. The final VLIW code is simulated, producing execution results. Our benchmarks are composed of seven nontrivial integer programs listed in Table I. The resource constraint of the VLIW machine is 16-ALUs and 8-way branching. The machine is assumed to have 32 general-purpose registers, 16 condition registers, perfect caches, and unit latencies.

EPS performs code motion pipelining. It renames registers using copies, on an as-needed basis when parallelism opportunities arise. For example, when $x = x + 4$ cannot be moved up due to its target register x (e.g., x is live at the other

Table I. Benchmark Description

Benchmarks	Lines	Descriptions
eqntott	3616	Truth table generation from equation
espresso	13639	Logic table optimization
li	7481	A lisp interpreter written in C
compress	1421	File compression utility
yacc	6482	A parser generator
sed	3289	A stream editor
gzip	7388	GNU file compression utility

target of a branch when we want to move the instruction speculatively above the branch), its target is renamed to `x'` and a copy instruction `copy x=x'` is left at the original place of the instruction. If the copy can later be eliminated, then this is equivalent to live range renaming; if not, it represents a more general way to get rid of nontrue data dependences for instruction scheduling.

In the context of EPS, some of these copies are generated to avoid cross-iteration register overwrites in software pipelining [Lam 1988] and are not coalescible due to interferences in the code. In order to remove those interferences, we unroll the loop which makes many of those copies coalescible [Kim et al. 2002]. After the unrolling, we perform register allocation and coalescing which yields better code with fewer copies. Copy elimination reduces resource pressure, enabling post-pass peephole compaction of adjacent VLIWs. More importantly, it obviates stalls caused by renaming multi-latency instructions in EPS[3] [Kim et al. 2002].

6.2 Coalescing and Spill Results

We have implemented the four coalescing heuristics (conservative, iterated, aggressive, and optimistic) on top of Briggs' optimistic allocator. Aggressive coalescing has the same phase ordering as optimistic coalescing in Figure 6, except that optimistic coalescing prefers the node with more ingredient nodes for the potential spill if two nodes have the same spill preference, and optimistic coalescing splits coalesced nodes in the select phase. We have also implemented extended optimistic coalescing described in Section 5, which we call *optimistic*$^+$ coalescing, as depicted in Figure 8. In optimistic$^+$ coalescing, a node with more ingredient nodes is preferred for the potential spill as in optimistic coalescing. As a "base-case" allocator, we have also implemented Chaitin's original allocator in Figure 1(a). In all allocators, we do not consider the call-cost [Lueh and Gross 1997], and regard a procedure call as a definition of all caller-save registers. Table II summarizes the configuration of each heuristic.

Figure 9 depicts the ratio of the number of copies removed by each heuristic to that of copies removed by the base-case allocator (i.e., 100% means the copies removed by Chaitin's allocator). The graph shows that conservative, iterated, aggressive, optimistic, and optimistic$^+$ coalescing remove a geometric mean of 60.0%, 75.5%, 99.0%, 98.9%, and 99.0% of the copies removed by Chaitin's,

[3]This is due to its code motion pipelining on a single loop body. Unroll-based copy elimination in EPS corresponds to *modulo variable expansion* in modulo scheduling [Lam 1988], except that it is applicable to loops with arbitrary control flows.

Table II. Configuration of Heuristics Experimented

Heuristic	Simplification style	Coalescing style	Undo-coalescing	Non-copy coalescing
base-case	pessimistic	aggressive	No	No
conservative	optimistic	conservative	No	No
iterated	optimistic	iterated	No	No
aggressive	optimistic	aggressive	No	No
optimistic	optimistic	aggressive	Yes	No
optimistic$^+$	optimistic	aggressive	Yes	Yes

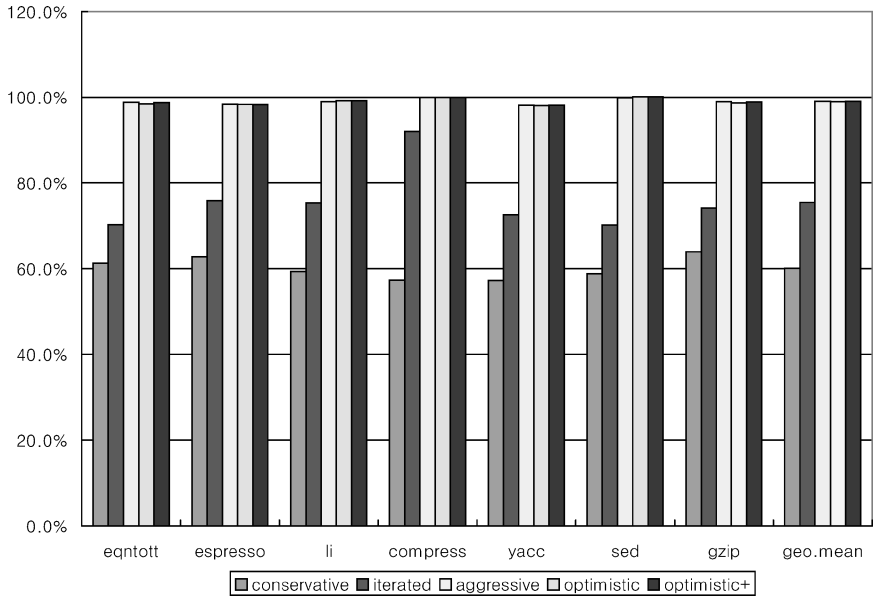

Fig. 9. Ratio of eliminated copies compared to those of the base-case allocator.

respectively. Optimistic coalescing eliminates almost the same number of copies as aggressive coalescing, and the difference of 0.1% is mostly due to the undo of coalescing via coalesced node splitting (except that the different ordering of nodes on the stack may result in a different coloring). Optimistic$^+$ coalescing eliminates slightly more or fewer copies compared to optimistic coalescing because noncopy coalescing produces a different result of coalesced node splitting.

One thing to note from the graph is that aggressive coalescing on top of Briggs' allocator removes an average of 1.0% fewer copies than on top of Chaitin's allocator. This is due to the fact that Chaitin's allocator spills more copy-related nodes which makes otherwise uncoalescible copies coalescible. For example, consider a copy $x = y$ which is not coalescible in the first round of both Chaitin's and Briggs' allocators because the live ranges of x and y interfere (e.g., there is a definition of y right after the copy). If Chaitin's allocator spills x while Briggs' does not, Chaitin's might be able to coalesce the copy in its second round because x is split into smaller pieces, making x and y in $x = y$ not interfere any

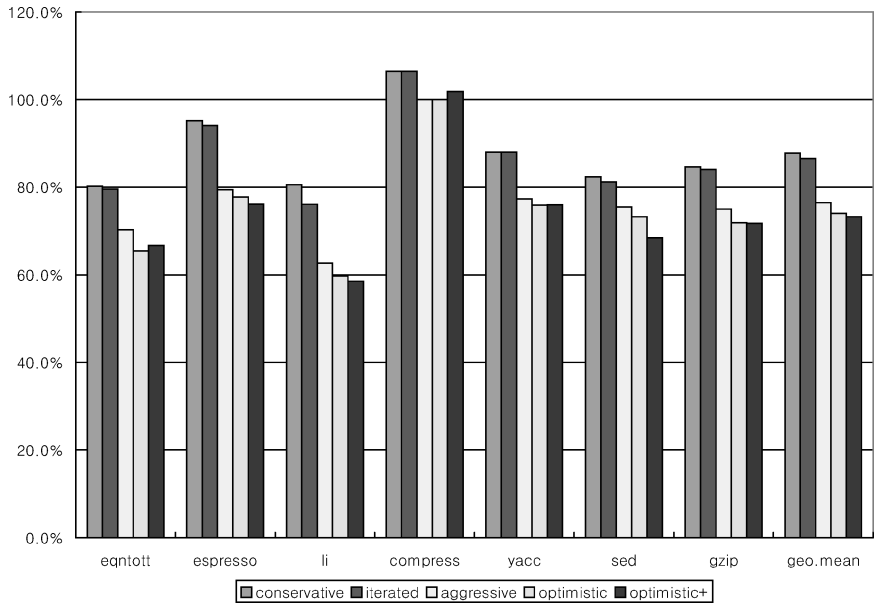

Fig. 10. Ratio of spill instructions compared to those of the base-case allocator.

more. That is, Chaitin's allocator eliminates more copies than Briggs' at the expense of more spills.

Figure 10 shows the ratio of the number of the spill instructions generated by each heuristic to that of spill instructions generated by Chaitin's (again, 100% means the spill instructions generated by Chaitin's). The graph shows that optimistic coalescing generates a smaller number of spill instructions than conservative coalescing or iterated coalescing in all benchmarks, and moreover, aggressive coalescing by itself is better than conservative coalescing or iterated coalescing.[4] The average difference between aggressive coalescing and optimistic coalescing is 2.5%, which is mainly due to the advantage in coloring caused by undoing coalescings.

Finally, optimistic$^+$ coalescing generates the smallest number of spill instructions in many benchmarks, which is 0.8% fewer than optimistic coalescing on average. This indicates the effectiveness of noncopy coalescing in improving the colorability of the graph.

6.3 Evaluation of Spill Results

The spill results indicate that in aggressive coalescing, the positive impact of coalescing outweighs the negative impact, improving the overall colorability of the graph. In this subsection, we analyze the experimental results in detail to confirm this by estimating the positive impact and the negative impact.

[4]In `compress`, conservative or iterated coalescing generate more spill instructions even than the base-case allocator. We found that the positive impact of coalescing in Chaitin's allocator out-weighs even the benefit of optimistic allocation.

Table III. Numbers of Nodes in Coalesced Chunks and Violating Chunks, and Actually-Spilled Nodes

Benchmark	Num. of Nodes in Coalesced Chunks (Num. of Chunks)	Num. of Nodes in Violating Chunks (Num. of Chunks)	Aggressive		Optimistic	
			Num. of Spilled Nodes (Chunks)	Ratio %	Num. of Spilled Nodes (Chunks)	Ratio %
eqntott	1634 (549)	1236 (389)	296 (70)	23.9	240 (70)	19.4
espresso	12250 (3976)	10293 (3165)	3041 (729)	29.5	2606 (754)	25.3
li	1369 (454)	964 (289)	198 (49)	20.5	179 (51)	18.6
compress	104 (42)	78 (29)	3 (1)	3.8	3 (1)	3.8
yacc	5265 (1681)	4310 (1321)	1008 (235)	23.4	876 (237)	20.3
sed	1105 (377)	769 (242)	123 (22)	16.0	105 (23)	13.7
gzip	4265 (1310)	3613 (1069)	1207 (292)	33.4	922 (290)	25.5
average				18.3		15.8

(a) Aggressive coalescing and optimistic coalescing

	Benchmark	Num. of Nodes in Coalesced Chunks (Num. of Chunks)	Num. of Nodes in Violating Chunks (Num. of Chunks)	Nodes (Chunks) Num. of Spilled	Ratio %
Chunks with Copy-Related Nodes Only	eqntott	1482 (500)	1084 (340)	240 (66)	22.1
	espresso	10837 (3468)	8880 (2657)	2571 (736)	29.0
	li	1263 (409)	858 (244)	175 (48)	20.4
	compress	91 (36)	65 (23)	3 (1)	4.6
	yacc	4495 (1406)	3540 (1046)	867 (234)	24.5
	sed	933 (317)	597 (182)	99 (22)	16.6
	gzip	3888 (1164)	3236 (923)	916 (291)	28.3
	average				18.3
Chunks with Non-Copy-Related Nodes	eqntott	265 (71)	257 (67)	4 (1)	1.6
	espresso	2979 (690)	2959 (680)	57 (15)	1.9
	li	220 (51)	220 (51)	4 (1)	1.8
	compress	24 (7)	24 (7)	0 (0)	0.0
	yacc	1417 (328)	1409 (324)	20 (5)	1.4
	sed	268 (55)	268 (55)	0 (0)	0.0
	gzip	849 (214)	849 (214)	10 (3)	1.2
	average				1.6

(b) Optimistic$^+$ coalescing

6.3.1 *Estimation of Negative Impact.* We first estimate the negative impact of coalescing by measuring how many of the nodes that have been coalesced in violation of the conservativeness heuristic are indeed actually-spilled to memory. This will show whether aggressive coalescing that delays the spill consideration to later can be justified. To avoid confusion in the following discussion, we use the term *node* for a single primitive node that has been created in the first build phase, and the term *chunk* for a coalesced node composed of several nodes.

In Table III(a), the first column shows the number of all coalesced chunks that are pushed on the stack in the first round of Briggs' optimistic allocator with aggressive coalescing or optimistic coalescing and the number of nodes in those chunks. Many of those chunks have been formed in violation of the conservativeness heuristic, and the number of violating chunks and the number of nodes in violating chunks are shown in the next column. It should be noted that these numbers are the same for both heuristics although the order of nodes on the stack may be different. Among those nodes in violating chunks,

the number of nodes (chunks) that are actually-spilled to memory by aggressive coalescing and by optimistic coalescing are described in the third and the fourth columns, respectively. The ratio column of those columns shows the ratio of the number of actually-spilled nodes to the number of nodes in violating chunks.

In aggressive coalescing, an average of 18.3% of nodes among those that have been coalesced nonconservatively are actually-spilled to memory. This means that for those remaining 81.7% of nodes, conservative coalescing would miss the coalescing opportunity even though they can be colored successfully after being coalesced. The spill rate further drops down to 15.8% in optimistic coalescing with the help of splitting coalesced nodes (which will be analyzed in detail shortly).

For optimistic$^+$ coalescing, we need to isolate the spill rate for those chunks that include non-copy-related nodes. In Table III(b), there are two subtables that correspond to those chunks composed of copy-related nodes only (the top table) and to those chunks that include one or more non-copy-related nodes (the bottom table). Since optimistic$^+$ coalescing coalesces copy-related nodes prior to non-copy-related nodes, it initially produces the same number of copy-related chunks as in Table III(a), but some of them are subsequently coalesced with non-copy-related nodes, so they are included in the bottom subtable. The bottom table indicates that optimistic coalescing is also advantageous for coalescing non-copy-related nodes; only an average of 1.6% of nodes in violating chunks are actually-spilled.

We now investigate the impact of coalesced node splitting in optimistic and optimistic$^+$ coalescing. Among those violating chunks in Table III (second columns), the first columns in Table IV show the number of chunks that have been the candidate of actual-spills (hence being split) and the number of nodes in those chunks.[5] A candidate chunk belongs to one of the following three categories after being split:

—Fully-spilled, meaning that all of its splits are spilled to memory.

—Fully-colored, meaning that all of its splits are colored successfully.

—Partially-spilled/colored, meaning that some splits are spilled while the rest are colored.

Table IV(a) shows the number of fully-spilled, fully-colored, and partially-spilled/colored chunks, and the number of nodes in those chunks in optimistic coalescing.[6] The ratio of successfully colored nodes to all split candidate nodes is an average of 17.1%, while the ratio of successfully split chunks (i.e., at least one of whose splits are colored) to all candidate chunks is an average of 49.4%. It should be noted that the number of successfully colored nodes here do not indicate the number of colorable primitive nodes. Since we try to minimize the

[5]It should be noted that the set of these actual-spill candidate chunks during the select phase is always a subset of potentially-spilled chunks during the simplify phase, which is also a subset of violating chunks.

[6]In optimistic and optimistic$^+$ coalescing, the number of spilled nodes (and chunks) in Table III is the sum of spilled nodes (and chunks) in partially-spilled and fully-spilled chunks in Table IV.

Table IV. Detailed Data of Undoing Coalescing in Optimistic and Optimistic$^+$ Coalescing

Bench-mark	Candidate Nodes (Candidate Chunks)	Spilled Nodes: in Fully Spilled Chunks	Spilled Nodes: in Partially Spilled/Colored Chunks	Colored Nodes: in Partially Spilled/Colored Chunks	Colored Nodes: in Fully Colored Chunks	Successful Coloring Ratio (%)	Successful Split Ratio (%)
eqntott	296 (70)	120 (29)	120 (41)	56 (41)	0 (0)	18.9	58.6
espresso	3129 (757)	1629 (397)	977 (357)	515 (357)	8 (3)	16.7	47.6
li	204 (51)	141 (38)	38 (13)	25 (13)	0 (0)	12.3	25.5
compress	3 (1)	3 (1)	0 (0)	0 (0)	0 (0)	0.0	0.0
yacc	1025 (238)	599 (136)	277 (101)	146 (101)	3 (1)	14.5	42.9
sed	127 (23)	35 (7)	70 (16)	22 (16)	0 (0)	17.3	69.6
gzip	1233 (296)	420 (94)	502 (196)	298 (196)	13 (6)	25.2	68.2
average						17.1	49.4

(a) Optimistic coalescing

	Bench-mark	Candidate Nodes (Candidate Chunks)	Spilled Nodes: in Fully Spilled Chunks	Spilled Nodes: in Partially Spilled/Colored Chunks	Colored Nodes: in Partially Spilled/Colored Chunks	Colored Nodes: in Fully Colored Chunks	Successful Coloring Ratio (%)	Successful Split Ratio (%)
Chunks with Copy-Related Nodes Only	eqntott	291 (67)	121 (29)	119 (37)	49 (37)	2 (1)	16.8	55.2
	espresso	3057 (740)	1672 (403)	899 (333)	477 (333)	9 (4)	15.6	45.0
	li	196 (48)	139 (37)	36 (11)	21 (11)	0 (0)	10.7	22.9
	compress	3 (1)	3 (1)	0 (0)	0 (0)	0 (0)	0.0	0.0
	yacc	1011 (236)	575 (132)	292 (102)	138 (102)	6 (2)	13.6	43.2
	sed	121 (22)	53 (10)	46 (12)	22 (12)	0 (0)	18.2	54.5
	gzip	1225 (296)	398 (89)	518 (202)	299 (202)	10 (5)	24.4	68.2
	average						16.0	45.7
Chunks with Non-Copy-Related Nodes	eqntott	6 (1)	0 (0)	4 (1)	2 (1)	0 (0)	33.3	100.0
	espresso	79 (15)	26 (5)	31 (10)	22 (10)	0 (0)	27.8	66.7
	li	4 (1)	4 (1)	0 (0)	0 (0)	0 (0)	0.0	0.0
	compress	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0.0	0.0
	yacc	36 (5)	2 (1)	18 (4)	16 (4)	0 (0)	44.4	80.0
	sed	0 (0)	0 (0)	0 (0)	0 (0)	0 (0)	0.0	0.0
	gzip	14 (3)	3 (1)	7 (2)	4 (2)	0 (0)	28.6	66.7
	average						33.0	77.2

(b) Optimistic$^+$ coalescing

spill cost in finding the most beneficial set of colorable primitive nodes, it may be the case that colorable nodes are actually spilled to prevent leaving splits on frequently executed paths.

One thing to note is that the number of actual-spill candidate chunks in optimistic coalescing in Table IV(a) is consistently larger than (or equal to in `compress` and `eqntott`) the number of actually-spilled chunks in aggressive coalescing in Table III(a). Since optimistic coalescing prefers coalesced nodes for the potential spill, coalesced nodes are more likely to be actual-spill candidates, compared to aggressive coalescing. Another reason is the constraint of undoing coalescing. Generally, if we split a coalesced node `xy` and color some of its splits, we constrain the coloring of other potentially-spilled nodes below it in the stack, making more of them actual-spill candidates, compared to just spilling `xy` in aggressive coalescing. The reason why optimistic coalescing spills less overall, despite these increased actual-spill candidate chunks, is that it can color many of the nodes in those chunks, as we have seen in Table IV(a).

Table IV(b) shows the result for optimistic$^+$ coalescing where the top and bottom subtables correspond to those subtables in Table III(b). An average of 33.0% of nodes in actual-spill candidate chunks that include non-copy-related nodes are assigned colors successfully.

Table V. Reduction Ratio of Interference Edges due to the Positive Impact of Coalescing

Benchmarks	Number of edges	Conservative	Iterated	Aggressive Optimistic	Optimistic$^+$
eqntott	60901	1.5%	2.6%	14.7%	19.6%
espresso	520138	1.0%	3.1%	19.5%	30.0%
li	67634	1.3%	2.4%	9.1%	11.6%
compress	5133	0.4%	5.0%	16.9%	22.2%
yacc	207035	1.3%	3.8%	19.3%	26.0%
sed	46526	2.0%	3.7%	14.4%	19.0%
gzip	159744	1.2%	3.0%	18.8%	26.1%
geo.mean		1.1%	3.3%	15.7%	21.2%

6.3.2 *Estimation of Positive Impact.* The above results indicate that the negative impact of coalescing can be alleviated by optimistic allocation, undoing coalescing, and the positive impact of coalescing. In order to analyze the positive impact of coalescing, we need to measure the number of successfully colored nodes due to the positive impact that would otherwise be spilled to memory. Unfortunately, this number is not easy to measure. Instead, we have measured the reduction of the interference edges due to the positive impact, as depicted in Table V. For each benchmark, the first column shows the number of interference edges right after the first build phase. The rest of the table shows the ratio of removed edges due to the positive impact for each heuristic, just before the first select phase. The table shows that aggressive (optimistic) coalescing removes nearly five times as many interference edges as iterated coalescing does on average. It also shows that optimistic$^+$ coalescing removes an average of 5.5% more interference edges.

6.4 Performance Impact

After register allocation and coalescing, we simulate the final VLIW code in order to measure its execution cycles. We compare the VLIW execution cycles of each heuristic to evaluate its performance impact. Since we have implemented each heuristic focusing only on reducing spills or removing copies without paying any attention to the spill effect on the VLIW schedule, the performance will be somewhat arbitrary, not necessarily being proportional to the static results such as the spill amount and the number of removed copies. In fact, it is an interesting research problem to choose appropriate spill nodes that do not destroy a given parallel schedule, but it is beyond the scope of this article and is thus not discussed here. Consequently, we report the performance impact of each heuristic with arbitrary spills.

Figure 11 shows the ratio of execution cycles of each heuristic compared to the base-case allocator (100% means the execution cycles of Chaitin's). The graph shows that optimistic and optimistic$^+$ coalescing outperform the base-case allocator even with arbitrary spills, by a geometric mean of 3.5% and 3.7%, respectively. However, the result is not always proportional to the ratio of removed copies in Figure 9 or the spill ratio in Figure 10. In some benchmarks (e.g., compress and li), it is rather proportional to the dynamic frequency of

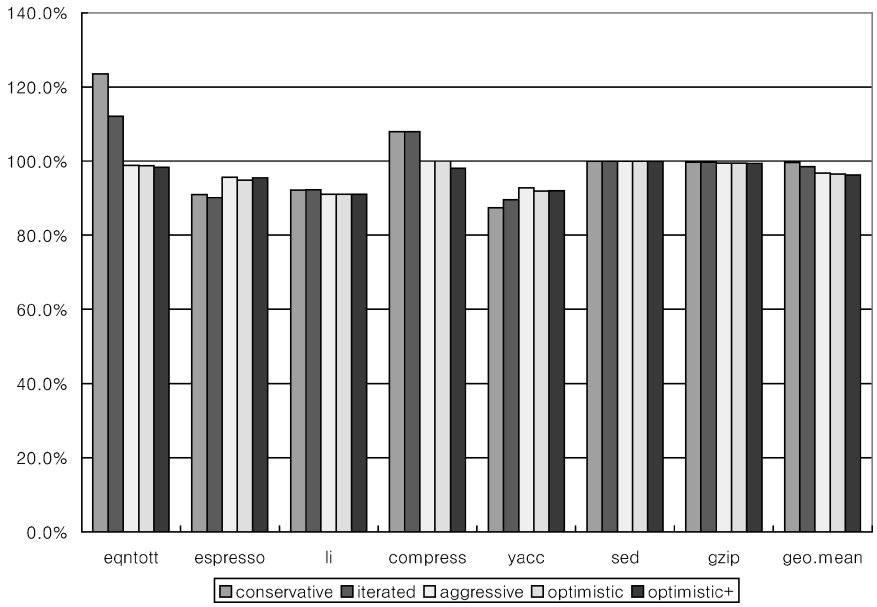

Fig. 11. Ratio of VLIW execution cycles compared to the base-case allocator.

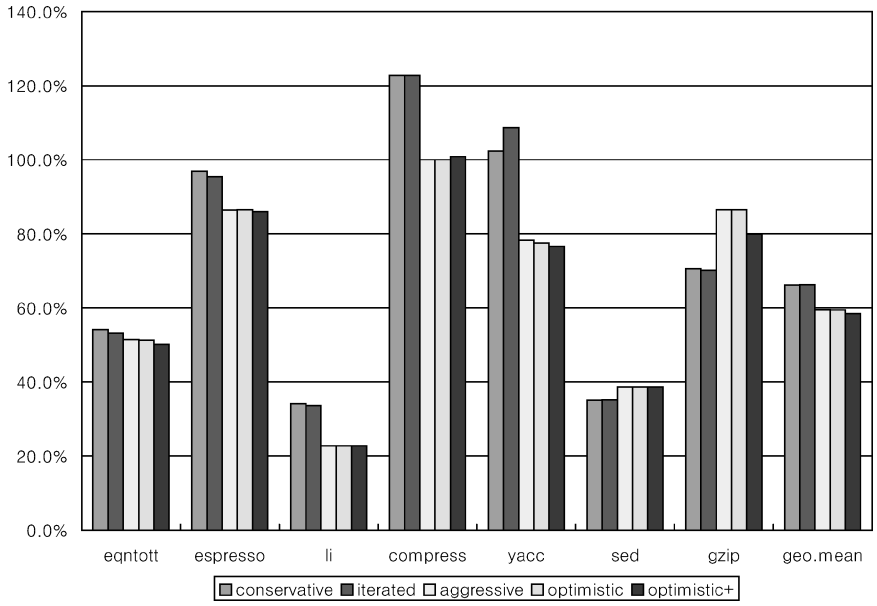

Fig. 12. Ratio of dynamic spill instructions compared to the base-case allocator.

spill instructions in Figure 12, which shows the ratio of dynamic spill instructions compared to the base-case allocator (again, 100% means the dynamic spill instructions of Chaitin's). However, there is no direct correlation between the execution cycles and the dynamic spill instructions in other benchmarks because the execution cycles are measured on a VLIW machine where arbitrary spill

instructions create arbitrary number of VLIW instructions, and the dynamic spill instructions are not dominant in the execution cycles in some benchmarks. For example, although conservative coalescing and iterated coalescing execute much fewer spill instructions in `sed` and `eqntott` than the base-case allocator, their execution cycles are comparable to or larger than that of base-case allocator in Figure 11 since the dynamic spill instructions are very small compared to the execution cycles. In `eqntott`, base-case allocator shows better performance than conservative coalescing and aggressive coalescing, because the execution cycles are rather heavily dependent on the number of removed copies in the most frequent loop.

The performance impact would be higher when we have latencies in the code since undeleted copies in EPS-scheduled code are likely to cause stalls [Kim et al. 2002]. On such an environment, the performance impact of copy elimination would be higher.

7. LIVE RANGE SPLITTING AND PRECOLORING

In this section, we describe how optimistic coalescing deals with two nontrivial register allocation issues, that is, *live range splitting* and *precoloring*, which have been the motivating issues for conservative coalescing and iterated coalescing, respectively.

7.1 Live Range Splitting and Optimistic Coalescing

Live range splitting is a technique for reducing the spill amount [Chow and Hennessy 1990; Kolte and Harrold 1993; Cooper and Simpson 1998; Bergner et al. 1997; Briggs 1992; Lueh 1997; Appel and George 2001]. It splits long live ranges into shorter ones by inserting copies and load/stores at appropriate places. Live range splitting may reduce the spill amount since shorter live ranges are more likely to be colored than longer ones. The cost involved with live range splitting is the additional instructions inserted for splitting, which must be cheaper than the spill instructions.

Several heuristic approaches have been taken for live range splitting on graph coloring register allocation. They can be roughly categorized into the *during*-coloring approach and the *before*-coloring approach, depending on when the splitting takes place [Lueh 1997]. The during-coloring approach splits uncolorable live ranges on demand during the coloring, generally in the color select phase [Chow and Hennessy 1990; Bergner et al. 1997; Cooper and Simpson 1998]. On the other hand, the before-coloring approach splits live ranges in advance before the coloring starts [Briggs 1992; Appel and George 2001].

The before-coloring approach might overly split live ranges since it cannot be known in advance which live ranges would need to be spilled. Therefore, a cleanup pass is required to remove unnecessary splits. This is the place where optimistic coalescing can be useful.

Briggs [1992] proposed several before-coloring live range splitting techniques in his dissertation. One observation was that live range splitting yields excessive copies which may be a bottleneck in increasing the performance. Although his conservative coalescing together with biased coloring in the cleanup phase

can remove copies, hence unproductive splits, many of them would still remain. Optimistic coalescing could be useful for this cleanup purpose, if combined with a good heuristic, due to its aggressive coalescing and its undoing capability, which would reduce unproductive splits.

Optimistic coalescing can also be useful for the cleanup phase of the *rematerialization*, which is a yet another before-coloring splitting technique [Briggs et al. 1994]. Optimistic coalescing obviously can remove useless split copies better than Briggs' coalescing phase due to its aggressive coalescing, but it can also keep the chance of cheap spilling achieved by rematerialization. That is, when aggressive coalescing of split copies results in actual spills, we try to spill rematerializable nodes. This can be handled as follows. When a potential spill is required, we prefer coalesced chunks that contain rematerializable nodes. When actual spills are required for those chunks, we try to split them in a way to isolate rematerializable nodes (this can be done simply by assigning a low spill cost to rematerializable nodes) and spill them first.

Appel and George's [2001] recent work on optimal spilling for "register-hungry" CISC machines is, in actuality, a good example for demonstrating the collaboration between before-coloring live range splitting and optimistic coalescing. They performed spilling first such that no more than K live ranges are simultaneously live at any program point, and then inserted splitting copies at "every" program point resulting in a K-colorable interference graph. To eliminate the excessive splitting copies, they employed optimistic coalescing because iterated coalescing was still too conservative for their purpose. It should be noted that the undo of coalescing can completely eliminate the negative impact of aggressive coalescing in this case since the interference graph is originally K-colorable. Because there are no spills in the coloring phase, Appel and George are able to use a much simpler version of our algorithm, omitting all the logic about how to split nodes instead of spilling.

7.2 Precoloring with Optimistic Coalescing

Some nodes in the interference graph should be assigned predetermined registers. For example, parameters must be passed in registers preallocated by the calling convention. These predetermined nodes must be *precolored* before other nodes are colored. This is simply represented in the interference graph by having K precolored nodes. These precolored nodes must be assigned an infinite spill cost because they should never be spilled.

Coalescing in the presence of precolored nodes may lead to an uncolorable graph or cause expensive spills. A precolored graph is uncolorable if there is a node n interfering with K nodes, each of which is precolored with a different color, while the node n itself cannot be spilled due to a lack of a temporary spill register. Coalescing may lead to this uncolorable graph by merging precolored nodes with normal nodes, converting normal neighbors into precolored ones. Even if coalescing does not actually produce such an uncolorable graph, it may make some cheap, normal nodes unspillable, causing more expensive spills.

Figure 13 illustrates this uncolorable graph problem when we employ aggressive coalescing. Let us assume that there are two registers `R1` and `R2`, both

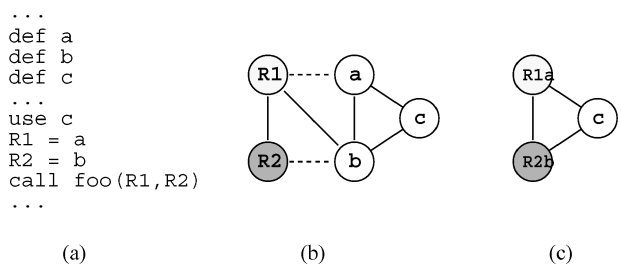

Fig. 13. Example of uncolorable graph after aggressive coalescing.

of which are used for parameter passing as shown in Figure 13(a). Figures 13(b) and 13(c) are the interference graphs before and after aggressive coalescing, respectively. The node `b` is assumed to have the smallest spill cost. The graph in Figure 13(c) is uncolorable because no node can be spilled; both `R1a` and `R2b` should not be spilled since they are already colored, while there is no temporary register for `c` to be spilled into. For successful coloring, either `a` or `b` should have been spilled instead of being coalesced.

Iterated coalescing avoids this problem by employing a special heuristic when coalescing a precolored node and a normal node [George and Appel 1996]. In Figure 13, for example, iterated coalescing would spill `b` before coalescing `R1` = `a`, thus obviating the problem. While this heuristic guarantees no change of the colorability of a graph, it may lose many opportunities for safe coalescing of precolored nodes since is still conservative as iterated coalescing itself.

Following the philosophy of optimistic coalescing, one might wish to perform aggressive coalescing of precolored nodes, potentially spill such a coalesced chunk if required, undo-coalesce the chunk when we have to actually spill it, and expect to spill only the nonprecolored split, exactly as we did with optimistic coalescing. Unfortunately, we cannot potentially spill such a precolored chunk since there is no guarantee that the same color is available for the precolored node when the chunk is split; the color of the precolored node may be taken by its interfering neighbor, if the neighbor is pushed on the stack later than the precolored node (hence being assigned a color earlier).

In order to avoid uncolorable graphs while allowing aggressive coalescing of precolored nodes, we implement the idea of optimistic coalescing differently for precolored nodes. When we coalesce precolored nodes during the coalescing phase, we do not merge them physically as we did for normal nodes; instead we achieve the effect of coalescing by adding some interference edges into the graph, as follows.

Coalescing a normal node m with a precolored node p means two actions: (1) assigning p's color to m and (2) making p's color unavailable for the neighbors of m. We achieve the effect of (1) by adding interference edges between m and all precolored nodes other than p. We achieve the effect of (2) by adding interference edges between p and all neighbors of m. Consequently, we make p's color be the only available one for m, without merging m and p physically. That is, they are "virtually" coalesced.[7] After the coalescing phase, the simplify phase proceeds

[7]If there is a node that interferes with both m and p, its degree can still be reduced by one although

as usual with these additional interference edges, and m can still be potentially spilled since it is not physically merged with p.

During the select phase, the interference edges of (2) are treated differently from regular edges, though. They represent *weak interferences* in the sense that we are allowed to dishonor them for a better colorability, without worrying about leading to illegal coloring (although we try to honor them to achieve coalescing). For example, if a node n that would otherwise need to be actually spilled has a weak interference edge with a precolored node p, we are allowed to color n with p's color to avoid the spill. In fact, if n is a neighbor of a node m and m was coalesced with p during the coalesce phase, and if m is still located on the stack, this violation would effectively undo the coalescing of m and p (since m cannot be colored by p's color), and make m uncolorable when it is popped out from the stack (since m cannot be colored by any other colors, either, due to interference edges of (1) with all other precolored nodes). We call such a node m a "victim" node since it is a victim for an actual spill. Our technique guarantees that violation of each weak interference edge leads to a victim node.

Due to weak interferences, we make two attempts to color a node during the select phase: first with those colors not used by any of its regular or weak neighbors, and second with those colors used by its weak neighbors. If multiple colors are available in the second attempt, we simply choose a color (and its weak interference edge) whose associated victim node has the smallest spill cost. If there exists a node that has already been selected as a victim node when processing a previous, popped-out node, we try to reselect it to reduce the overall spill cost since it will be actually spilled anyway.

When a victim node m is popped out from the stack, we do not have to actually spill it right away. Since coalescing is already undone at this point, the interference edges of (1) are meaningless and we can color m if there is a color available. However, coloring m at this point might ruin the rest of the coloring if there exists a node below m in the stack which has been simplified based on the fact that m and p would use the same colors. Therefore, we wait until all nodes in the stack are processed and then attempt to color m, ignoring the edges of (1). This is similar to what we did when both splits are colorable after undo-coalescing in Section 4.1.

Figure 14 illustrates how this technique solves the uncolorable graph problem of Figure 13. Figure 14(a) shows the coalescing phase and the interference graph after coalescing is done. The additional interferences are depicted with thick edges. It should be noted that `a` and `b` are not physically merged with `R1` and `R2`, respectively, and thus are still available for the potential spill. In the simplify phase, the cheapest node `b` is potentially spilled, and then `a` and `c` are simplified, while `R1` and `R2` remain in the graph.

Figure 14(b) shows the color selection phase. From the stack `c` is popped out first but no color is available for it at the first attempt. At the second attempt, both `R1` and `R2` are available for `c`. We select `R2` since the victim node `b` associated with it is cheaper. As we color `c` with `R2`, the coalesce of `b` and `R2` is effectively

they are virtually coalesced, since m can be colored only by the color of p. That is, we can still exploit the positive impact of coalescing.

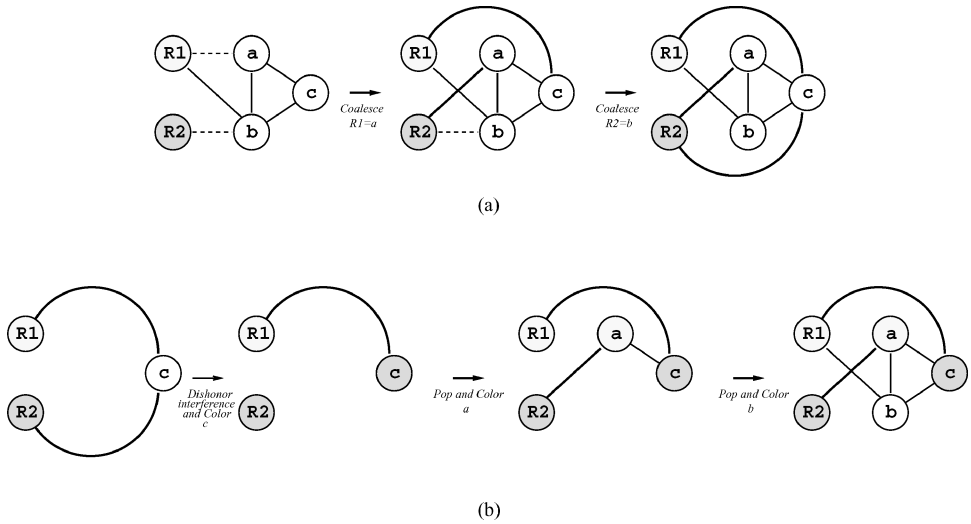

Fig. 14. Example of spilling nodes coalesced to precolored nodes.

undone, which leads to an actual spill of b when it is popped out. Consequently, the graph is colored successfully.

8. SUMMARY AND FUTURE WORK

In this article, we have proposed a new coalescing heuristic called optimistic coalescing that aggssively exploits the positive impact of coalescing by aggressive coalescing while alleviating its negative impact by optimistic allocation and undoing coalescing. Our experimental results indicate that compared to other heuristics, the new heuristic achieves the better overall result of copy elimination and spill reduction, which has also been confirmed by detailed analysis of the experimental data. We have also proposed optimistic coalescing of non-copy-related nodes using merge-enhanced graph coloring in order to exploit the positive impact further.

Since conservative coalescing and aggressive coalescing take completely different approaches, it is premature to conclude that optimistic coalescing is always the best, although it showed some promise of aggressive coalescing. It is left as a future work to evaluate optimistic coalescing in other environment including machines with fewer registers and improve it into a more general solution for coalescing and register allocation.

APPENDIX

A. PSEUDOCODE

This appendix provides the pseudocode for optimistic coalescing based on George and Appel's [1996] pseudocode for iterated coalescing. One of the advantages of their implementation is that it does not eliminate nodes from neighbor node lists in the coalesce phase, thus facilitates the efficient recovery of the original interference described in Section 4.2.2.

Many procedures from the original pseudocode are reused without any changes, but there are some modified procedures and new procedures for optimistic coalescing. This section describes the modified procedures and the new procedures only.

Reused `LivenessAnalysis`, `Build`, `AddEdge`, `Simplify`, `RewriteProgram`, `Adjacent`, `GetAlias`

Modified `Main`, `MkWorklist`, `DecrementDegree`, `Coalesce`, `Combine`, `SelectSpill`, `AssignColors`

New `OkColors`, `TryPrimitiveNodes`, `UndoCoalescing`, `FixAdj`

Most data structures are reused. The unused data structures and the new data structures are as follows.

Unused `freezeWorklist`, `frozenMoves`, `activeMoves`

New `originalAdjList`, `savedSpillCost`

The extended optimistic coalescing and precolored node coalescing described in Section 5 and Section 7.2 respectively are not included in this pseudocode.

```
procedure Main ()
  LivenessAnalysis()
  Build()
  forall t ∈ initial
      originalAdjList[t] := adjList[t]
      savedSpillCost[t] := spillCost[t]
  while worklistMoves ≠ {} do Coalesce()
  MkWorklist()
  repeat
      if simplifyWorklist ≠ {} then Simplify()
      else if spillWorklist ≠ {} then SelectSpill()
  until simplifyWorklist = {} ∧ spillWorklist = {}
  AssignColors()
  if spilledNodes ≠ {} then
      RewriteProgram(spilledNodes)
      Main()
```

Before coalescing, the original neighbor list of each node is saved in the `originalAdjList`. In the actual implementation, the `originalAdjList` is a pointer into the `adjList`. By making the procedure `AddEdge` add neighbors to the front of the `adjList`, we can separate the newly added neighbors and the original neighbors using the `originalAdjList`. Note that unlike the description in Section 4.2.2, we made the `originalAdjList` a pointer to the original neighbors rather than the newly added neighbors for the convenience of description.

```
procedure MkWorklist ()
  forall n ∈ initial
      initial := initial \ {n}
      if degree[n] ≥ K then
          spillWorklist := spillWorklist ∪ {n}
      else
          simplifyWorklist := simplifyWorklist ∪ {n}
```

```
procedure DecrementDegree (m)
  let d = degree[m]
  degree[m] := d-1
  if d = K then
      spillWorklist := spillWorklist \ {m}
      simplifyWorklist := simplifyWorklist ∪ {m}
```

In `MkWorklist` and `DecrementDegree`, all low-degree nodes are put in the `simplifyWorklist`, unlike iterated coalescing that puts copy-related low-degree nodes in the `freezeWorklist`.

```
procedure Coalesce ()
  let m(=copy(x,y)) ∈ worklistMoves
  x := GetAlias(x)
  y := GetAlias(y)
  if y ∈ precolored then
      let (u,v) = (y,x)
  else
      let (u,v) = (x,y)
  worklistMoves := worklistMoves \ {m}
  if u = v then
      coalescedMoves := coalescedMoves ∪ {m}
  else if v ∈ precolored ∨ (u,v) ∈ adjSet then
      constrainedMoves := constrainedMoves ∪ {m}
  else if u ∈ precolored then
      Coalesce precolored nodes carefully as described in Section 7.2.
  else
      coalescedMoves := coalescedMoves ∪ {m}
      Combine(u,v)

procedure Combine (u,v)
    coalescedNodes := coalescedNodes ∪ {v}
    alias[v] := u
    forall t ∈ Adjacent(v)
        AddEdge(t,u)
        if t ∉ precolored then
            degree[t] := degree[t] - 1
```

Procedure `Combine` cannot use the procedure `DecrementDegree`, since it updates the `simplifyWorklist` and `spillWorklist` yet to be set up.

```
procedure SelectSpill()
  let m ∈ spillWorklist selected using favorite heuristic
      Note: Choose node with more ingredient nodes
      if two nodes are equally preferred by the heuristic.
      Avoid choosing nodes that are the tiny live ranges
      resulting from the fetches of previously spilled registers.
  spillWorklist := spillWorklist \ {m}
  simplifyWorklist := simplifyWorklist ∪ {m}

function OkColors (n)
    let okColors = {0, ..., K-1}
    forall w ∈ Adjacent(n)
        if w ∈ (coloredNodes ∪ precolored) then
            okColors := okColors \ {color[w]}
    return okColors
```

```
procedure AssignColors ()
    while selectStack not empty
        let n = pop(selectStack)
        let okColors = OkColors(n)
        if okColors ≠ {} then
            coloredNodes := coloredNodes ∪ {n}
            let c ∈ okColors
            color[n] := c
        else if ∃v ≠ n, GetAlias(v) = n then
            TryPrimitiveNodes(n)
        else
            spilledNodes := spilledNodes ∪{n}
    forall n ∈ coalescedNodes
        color[n] := color[GetAlias(n)]
procedure TryPrimitiveNodes (n)
    let primitiveNodes = UndoCoalescing(n)
    let colorableNodes = {}
    forall v ∈ primitiveNodes
        let okColors = OkColors(v)
        if okColors ≠ {} then
            colorableNodes := colorableNodes ∪ {v}
        else
            spilledNodes := spilledNodes ∪ {v}
    forall C ⊂ colorableNodes in a spill cost order
        if (∀t ∈ C, ∃ c ∈ {0,...,K-1}, s.t. (t, c) ∉ adjSet) then
            forall t ∈ C
                coloredNodes := coloredNodes ∪ {t}
                color[t] := c
            forall t ∈ (colorableNodes \ C)
                place t at the bottom of selectStack
            return function UndoCoalescing (n)

    let primitiveNodes = {v| GetAlias(v) = n}
    forall v ∈ primitiveNodes
        FixAdj(v)
        spillCost[v] := savedSpillCost[v]
        coalescedNodes := coalescedNodes \ {v}
        alias[v] := v
    return primitiveNodes
```

Function `UndoCoalescing` returns the primitive nodes of a given coalesced node with the necessary information such as the neighbor lists and the spill costs being recovered.

```
procedure FixAdj (n)
    forall v ∈ adjList[n] \ originalAdjList[n]
        if v ∉ precolored then
            adjList[v] := adjList[v] \ {n}
        adjSet := adjSet \ {(v, n), (n, v)}
    adjList[n] := originalAdjList[n]
    forall v ∈ adjList[n]
        AddEdge(GetAlias(v), n)
```

Procedure `FixAdj` fixes the interference graph for a given primitive node as described in Section 4.2.2. The `degree` is not used anymore, so it is not required to keep it correct while removing and adding interferences in `FixAdj`.

Compared to iterated coalescing, optimistic coalescing does not have the runtime overhead of examining the neighbors' degrees to test the conservative coalescing heuristic or maintaining a worklist of freezable copies. However, there is an additional overhead of undo-coalescing and coloring splits.

REFERENCES

APPEL, A. AND MACQUEEN, D. 1991. Standard ML of New Jersey. In *Proceedings of the 3rd International Symposium on Programming Language Implementation and Logic Programming*.

APPEL, A. W. AND GEORGE, L. 2001. Optimal spilling for CISC machines with few registers. In *Proceedings of the 2001 ACM SIGPLAN Conference on Programming Languages Design and Implementation.* ACM, New York, 243–253.

BERGNER, P., DAHL, P., ENGEBRETSEN, D., AND O'KEEFE, M. 1997. Spill code minimization via interference region spilling. In *Proceedings of the 1997 ACM SIGPLAN Conference on Programming Languages Design and Implementation.* ACM, New York, 287–295.

BRIGGS, P. 1992. Register allocation via graph coloring. Ph.D. dissertation, Rice University.

BRIGGS, P., COOPER, K. D., AND TORCZON, L. 1994. Improvements to graph coloring register allocation. *ACM Trans. Prog. Lang. Syst. 16*, 3 (May), 428–455.

CHAITIN, G. 1982. Register allocation and spilling via graph coloring. In *Proceedings of the ACM SIGPLAN 1982 Symposium on Compiler Construction.* ACM, New York, 201–207.

CHOW, F. C. AND HENNESSY, J. L. 1990. The priority-based coloring approach to register allocation. *ACM Trans. Prog. Lang. Syst, 12*, 4 (Oct.), 501–536.

COOPER, K. D. AND LU, J. 1997. Register promotion in C programs. In *Proceedings of the 1997 ACM SIGPLAN Conference on Programming Languages Design and Implementation.* ACM, New York, 308–319.

COOPER, K. D. AND SIMPSON, L. T. 1998. Live range splitting in a graph coloring register allocation. In *Proceedings of the 1998 International Compiler Construction Conference*.

CYTRON, R., FERRANTE, J., ROSEN, B., WEGMAN, M., AND ZADECK, F. 1991. Efficiently computing static single assignment form and the control dependence graph. *ACM Trans. Prog. Lang. Syst. 13*, 4 (Oct.), 451–490.

GEORGE, L. AND APPEL, A. W. 1996. Iterated register coalescing. *ACM Trans. Prog. Lang. Syst. 18*, 3 (May), 300–324.

KIM, S., MOON, S.-M., PARK, J., AND EBCIOĞLU, K. 2002. Unroll-based copy elimination for enhanced pipeline scheduling. *IEEE Trans. Comput. 51*, 9 (Sept.), 977–994.

KOLTE, P. AND HARROLD, M. J. 1993. Load/store range analysis for global register allocation. In *Proceedings of the 1993 ACM SIGPLAN Conference on Programming Languages Design and Implementation.* ACM, New York, 268–277.

LAM, M. 1988. Software pipelining: An effective scheduling technique for VLIW machines. In *Proceedings of the 1988 ACM SIGPLAN Conference on Programming Languages Design and Implementation.* ACM, New York, 318–328.

LUEH, G.-Y. 1997. Fusion-based register allocation. Ph.D dissertation, Carnegie Mellon Univ.

LUEH, G.-Y. AND GROSS, T. 1997. Call-cost directed register allocation. In *Proceedings of the 1997 ACM SIGPLAN Conference on Programming Languages Design and Implementation.* ACM, New York, 296–307.

MOON, S.-M., CHUNG, H., PARK, J., SHIM, S., AND AHN, J.-W. 1998. SPARC-based VLIW testbed. *IEE Proc. Comput. Digital Tech.*, 215–224.

MOON, S.-M. AND EBCIOĞLU, K. 1997. Parallelizing nonnumerical code with selective scheduling and software pipelining. *ACM Trans. Prog. Lang. Syst. 19*, 6 (Nov.), 853–898.

VEGDAHL, S. R. 1999. Using node merging to enhance graph coloring. In *Proceedings of the 1999 ACM SIGPLAN Conference on Programming Language Design and Implementation.* ACM, New York, 150–154.

Received February 2003; revised June 2003; accepted October 2003